

Universidad del Valle de Guatemala
Ingeniería en Ciencias de la Computación
Curso: Programación orientada a objetos



Análisis y diseño programa
Control de Órdenes de Servicio de un Taller Automotriz

Roger Yorkaef Méndez García
26786
25/08/2026

Análisis

Clases

- OrdenServicio - M
- VistaConsola - V
- SistemaTaller- C
- Main - C

Model - azul
View - verde
Controller - rojo

Clase: OrdenServicio

Propiedades

Atributo	Tipo	Visibilidad	Función
numeroOrden	int	private	Guarda el número que identifica a la orden de servicio en el sistema
nombrePropietario	String	private	Guarda el nombre del propietario del vehículo asociado a la orden
placaVehiculo	String	private	Guarda la placa del vehículo asociado a la orden de servicio
descripcionServicio	String	private	Guarda la descripción del mantenimiento o reparación solicitado
costoEstimado	double	private	Guarda el costo estimado correspondiente al servicio solicitado

Métodos

Método	Parámetro	Retorno	Visibilidad	Función
OrdenServicio()	int numeroOrden, String nombrePropietari o, String placaVehiculo, String descripcionServic io, double costoEstimado	—	public	Inicializa una orden de servicio con el número, propietario, placa, descripción y costo estimado recibidos
getNumeroOrden()	Ninguno	int	public	Devuelve el número que tendrá la orden del servicio
getNombrePropietario ()	Ninguno	String	public	Devuelve el nombre del propietario registrado en la orden
getPlacaVehiculo ()	Ninguno	String	public	Devuelve la placa del vehículo que tendrá la orden
registrgetDescripcionServicio arTiempo ()	Ninguno	String	public	Devuelve la descripción del servicio registrado
getCostoEstimado ()	Ninguno	double	public	Devuelve el costo estimado de la orden
modificarServicio ()	String nuevaDescripcion, double nuevoCosto	boolean	public	Valida la nueva descripción y el nuevo costo. Si son válidos, modifica la información de la orden y devuelve true, si no, devuelve false

Clase: VistaConsola

Propiedades

Atributo	Tipo	Visibilidad	Función
scanner	Scanner	private	Recibir mediante consola las entradas dadas por el usuario

Métodos

Método	Parámetro	Retorno	Visibilidad	Función
VistaConsola ()	Ninguno	—	public	objeto utilizado para recibir las entradas del usuario mediante consola
mostrarMenu ()	Ninguno	int	public	Muestra las opciones disponibles y devuelve la opción que selecciona el usuario
solicitarNumeroOrden ()	Ninguno	int	public	Solicita y devuelve el número de una orden de servicio
solicitarNombrePropietario ()	Ninguno	String	public	Solicita y devuelve el nombre del propietario del vehículo
solicitarPlaca ()	Ninguno	String	public	Solicita y devuelve la placa del vehículo

solicitarDescripcion ()	Ninguno	String	public	Solicita y devuelve la descripción del servicio
solicitarCosto ()	Ninguno	double	public	Solicita y devuelve el costo estimado de una orden
mostrarOrden ()	OrdenServicio orden	void	public	Muestra en consola toda la información correspondiente a un servicio
mostrarMensaje ()	String mensaje	void	public	Muestra en la consola un mensaje recibido por parámetro
leerEntero ()	String mensaje	int	private	Solicita una entrada hasta recibir un valor entero, controlando entradas incorrectas con try-catch
leerDouble ()	String mensaje	double	private	Solicita una entrada hasta recibir un valor numérico decimal, controlando entradas incorrectas con try-catch

Clase: SistemaTaller

Propiedades

Atributo	Tipo	Visibilidad	Función
ordenes	List<OrdenServicio>	private	Almacena las órdenes de servicio activas registradas en el sistema. La colección será creada mediante una instancia de ArrayList
vista	VistaConsola	private	Solicitar información al usuario y mostrar los resultados de las operaciones por medio de la consola

Métodos

Método	Parámetro	Retorno	Visibilidad	Función
SistemaTaller ()	Ninguno	—	public	Inicia una colección dinámica vacía mediante ArrayList y crea la instancia de VistaConsola
iniciar ()	Ninguno	void	public	Mantiene activo el menú principal y la ejecución del sistema, hasta que el usuario seleccione “salir”

procesarOpcion ()	int opcion	void	private	Determina la operación que debe ejecutarse según la opción que sea seleccionada en el menú
registrarOrden ()	Ninguno	void	private	Solicita los datos necesarios, valida que sean correctos y que el número no esté registrado antes de agregar una nueva orden a la colección
consultarOrdenes ()	ninguno	void	private	Recorre el arreglo dinámico y muestra todas las órdenes registradas
buscarOrden ()	ninguno	void	private	Solicita un número de orden, realiza una búsqueda y muestra la información si existe
modificarOrden ()	ninguno	void	private	Solicita el número de orden, nueva descripción y nuevo costo, localiza la orden y solicita hacer la modificación
cancelarOrden ()	ninguno	void	private	Solicita un número de orden, localiza el objeto y lo elimina del arreglo dinámico

consultarOrdenesPorPlaca ()	ninguno	void	private	Solicita una placa y muestra todas las órdenes de servicio asociadas a esa
mostrarReporteCostos ()	ninguno	void	private	Muestra el costo total y el costo promedio de las órdenes activas
mostrarOrdenMayorCosto ()	Ninguno	void	private	Obtiene la orden que tiene costo estimado mayor y muestra toda su información
mostrarCantidadOrdenes ()	ninguno	void	private	Muestra la cantidad de órdenes en el arreglo
buscarOrdenPorNumero ()	int numeroOrden	OrdenServicio	private	Busca en el arreglo una orden con número que coincida con el solicitado y si lo encuentra muestra el objeto
buscarOrdenesPorPlaca ()	String placa	List<OrdenServicio>	private	Busca en el arreglo y devuelve una lista con todas las órdenes asociadas a la placa recibida
calcularCostoTotal ()	ninguno	double	private	Recorre las órdenes registradas y devuelve la suma de sus costos estimados

calcularCostoPromedio ()	ninguno	double	private	Calcula y devuelve el costo promedio de las órdenes activas
obtenerOrdenMayorCosto ()	ninguno	OrdenServicio	private	Busca en las órdenes y devuelve el objeto que corresponda a la orden con el costo estimado más alto
obtenerCantidadOrdenes ()	ninguno	int	private	Devuelve la cantidad de órdenes registradas

Clase: Main

Propiedades

- Esta clase no cuenta con atributos porque su responsabilidad consiste únicamente en funcionar como punto de entrada del programa.

Métodos

Método	Parámetro	Retorno	Visibilidad	Función
main()	String[] args	void	public static	Punto de entrada del programa. Inicia la ejecución del sistemaTaller

Preguntas:

- Las preguntas relacionadas directamente con clases, atributos, métodos, tipos, parámetros y modificadores de visibilidad se encuentran respondidas mediante las tablas anteriores .
4. ¿Qué información deberá almacenarse dentro de la colección dinámica?
- La colección almacenará objetos de tipo OrdenServicio, correspondientes a las órdenes activas registradas en el taller
5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?
- La colección será declarada como List<OrdenServicio> y será creada mediante ArrayList<OrdenServicio> en el constructor de SistemaTaller
8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
- Los objetos serán inicializados mediante sus constructores. SistemaTaller comenzará con una colección vacía y cada OrdenServicio recibirá sus datos al momento de ser creada.
9. ¿Cómo identificará una orden específica dentro de la colección?
- Se utilizará numeroOrden como identificador. La colección buscará hasta encontrar una orden cuyo número coincida con el solicitado.
10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?
- Las órdenes serán agregadas con add(). Y Para cancelar una orden, primero se encontrará el objeto y después se eliminará con remove().

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

- Se recorrerá toda la colección comparando las placas. Las que sean iguales serán almacenadas en una lista extra para devolver y después mostrar todas las órdenes encontradas.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

- Se recorrerán las órdenes sumando sus costos para obtener el total. El promedio se sacará dividiendo ese resultado entre la cantidad de órdenes, mientras que la colección no esté vacía.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

- Se recorrerá la colección comparando los costos y manteniendo una referencia con el mayor valor encontrado hasta ese momento.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

- Una entrada con un tipo de dato incorrecto puede producir una excepción. También debe controlarse el intento de buscar, modificar o cancelar una orden inexistente.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

- Se protegerá la lectura e ingreso de valores numéricos. De esta forma un error no termina la ejecución del programa.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

- El bloque finally mostrará un mensaje indicando que la operación ha finalizado. Se ejecutará tanto cuando la operación sea exitosa como cuando ocurra una excepción.